Why the LIMO Processor Simulator Does Not Work

CS3520 LIMO Group Project · Team Data-Academia · prepared 9 November 2026

Basis: limo-Data-Academia-main-1.zip. I compiled the C++ code with g++, ran the existing tests, ran the three M1 sample programs through both the single-cycle CPU and the pipeline, and ran the assembler functions from index.html under Node. Results below are from those runs.

Short answer

There are three separate problems, and the simulator cannot work until all three are fixed:

1. The web page is an assembler with placeholder tables. It does not simulate anything, and its assembler rejects its own demo program.
2. The C++ pipeline has no hazard unit (no forwarding, no load-use stall), so any program with dependent instructions gives wrong results.
3. Nothing connects the two: no program loader, no command-line entry point, and no way to run the pipeline from the page.

1. The web page (index.html) is not a simulator

No simulation code. The page has an editor, an Assemble button and a Load demo button. There are no Step, Run or Reset controls and no pipeline view. The title is “LIMO M1 Assembler”.

Placeholder tables. renderRegisters() writes 0x00000000 for r0–r15 and renderMemory() writes eight zero words. Neither is ever updated after assembling.

The assembler fails on the spec’s own programs. The parser accepts a register only as a bare number or with an x prefix (3, x3). It does not accept r3, so the page’s own default demo (eketsa r3, r1, 5 …) fails on load with “Bad register operand”. Results of my tests:

Input	Result
Page default demo (r-numbered registers)	Fails: Bad register operand
M1 Program 1 as written (n1, noto, aliases)	Fails: Bad register operand
Label on the same line as an instruction (loop: kopanya …)	Fails: Unable to parse line
eketsa x20, x0, 5 (register above r15)	Assembles to 0x00500A13, which sets a reserved bit, so the decoder treats it as a NOP
eketsa x3, x0, 5000 (outside −2048..2047)	Assembles silently to 0x38800193 with a wrapped immediate
lekanya x1, x2, +8000 (outside ±4096)	Assembles silently to 0xF42080E3 with a wrapped offset

Gaps against the M1 lexical rules. The assembler does not accept register aliases (noto, lehae, lesupa, n1–n6, k1–k4, p1–p3) or Sesotho names. It does not restrict registers to r0–r15, does not range-check I, S or branch immediates, and does not normalise apostrophes. Case-insensitive mnemonics and decimal/hex immediates do work.

2. The C++ pipeline has no hazard unit

Pipeline.h says so directly: forwarding and load-use stalls are “not included yet”, and dependent instructions need NOPs between them. IdEx keeps rs1, rs2 and rd “for the hazard unit”, but nothing reads them.

The existing pipeline test passes only because its programs are made of independent instructions. The M1 sample programs are full of dependencies. I ran each through the single-cycle CPU (the reference) and the pipeline:

Program	Expected (M1)	Single-cycle CPU	Pipeline
1: sum 1 to 5	r3 = 15, r4 = 0, Mem[0] = 15; 30 cycles	Correct (18 instructions)	Never finishes: hit the 2000-cycle cap with garbage in r3 and r4
2: three-element array	r3..r7 = 30, 12, 12, 60, 30; Mem = 10, 20, 30, 60; 33 cycles	Correct (22 instructions)	Wrong: r4 = 16, r6 = 30, Mem[0] = 0, finishes in 36 cycles
3: logic, shift, branch	r3..r9 = 12, 10, 8, 14, 6, 24, 24; Mem[0] = Mem[4] = 24; 17 cycles	Correct (10 instructions)	Wrong: r5 = 0, r6 = 12, r7 = 0, r8 = 0, r9 = 0, memory not written

The single-cycle results show the decoder, ALU, register file and memory are correct. The failure is in the pipeline control. Program 1 never ends because the loop counter is read before the previous eketsa has written it back, so the phapanyo exit test never sees 0.

What the hazard unit needs.

- **Forwarding unit.** Compare ID/EX rs1 and rs2 against the rd of the instruction in EX/MEM first, then MEM/WB (4-bit comparators, EX/MEM wins). Skip rd = r0 and instructions with RegWrite = 0.
- **Forwarding muxes.** Place them in front of the ALUSrc mux, and feed their output to the branch comparison and to the store data for beha (design doc section 2.1).
- **Load-use stall.** If the instruction in ID/EX is a load (MemRead) and its rd matches rs1 or rs2 of the instruction in IF/ID, hold the PC and IF/ID for one cycle and insert a bubble into ID/EX. This produces the one-cycle stall shown in M1 design-decision (d).
- **Cycle counts.** The M1 counts (30, 33, 17) assume full forwarding, one load-use stall and two flushed instructions per taken branch. They can only be reproduced once this unit exists.

3. Instruction and data share one memory

M1 specifies separate instruction and data memories and a 4 GB address space. The code uses a single 1 MiB array for both. M1 Programs 1 and 2 store to Mem[0], Mem[4] and Mem[8], which are the addresses of their own first instructions. The programs happen to survive in the single-cycle run because those instructions have already executed, but in a pipeline that prefetches, this is a real risk. Memory also throws on any address above 1 MiB, which would stop the simulation instead of behaving as the ISA describes.

4. Nothing ties the pieces together

- **No entry point.** CMake builds a static library and three test programs. There is no main(), so nothing loads a program, runs it, or prints the registers and memory.
- **No assembler-to-pipeline path.** Tests hand-encode machine words. The JavaScript assembler cannot hand its output to the C++ code, and there is no C++ assembler in this repo version.
- **No way to run C++ in the page.** The page needs either a JavaScript port of the pipeline or a WebAssembly build (for example with Emscripten) of the C++ classes, plus a small interface for load, step, run, reset and state dump.
- **Small mismatches.** The single-cycle CPU::step throws on illegal instructions and misaligned addresses instead of treating them as NOP and ignoring address bits [1:0]. Pipeline.cpp already does this correctly.

5. What is missing, in the order I would fix it

1. **Assembler.** Accept r0–r15, aliases and Sesotho names; allow a label and an instruction on one line; range-check registers and immediates; normalise apostrophes. Then test all three M1 programs.
2. **Hazard unit.** Add forwarding and the load-use stall to Pipeline.cpp, with tests for each hazard type (EX/MEM forward, MEM/WB forward, load-use, store data, branch operands).
3. **Regression tests.** Store Programs 1–3 as example files with expected registers, memory and cycle counts, and run them on every build.

4 **Memories.** Split instruction and data memory and widen the address space, or document the 1 MiB limit as a decision-log entry.

5 **Engine in the page.** Port or compile the pipeline and connect it to the assembler.

6 **Interface.** Add Step, Run and Reset, live register and memory views, the pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) and a cycle counter.